

Contents

Architecture Review — LedgerLite (fictional multi-tenant invoicing API)	1
Scope & method	1
Architecture at a glance	1
Top findings (Starter tier - 9 of 13 total; see the Standard/Advanced tier report for the complete finding set)	1
A-01 — Critical	1
A-02 — Critical	2
A-03 — High	2
A-04 — Critical	2
A-05 — High	3
A-06 — High	3
A-09 — High	3
A-12 — High	3
A-13 — High	4
Risk summary	4

Architecture Review — LedgerLite (fictional multi-tenant invoicing API)

Deliberately insecure for demonstration and training. Do not deploy. Run locally only.

Prepared: 2026-09-28 · **Tier:** Starter · **Target:** `targets/tenant-api` (Java 21 / Spring Boot 3 / PostgreSQL / Keycloak, shared-schema multi-tenancy)

Scope & method

A source-available review of LedgerLite's tenant-isolation model: how a request's tenant is resolved, how that resolution is (or isn't) enforced at the data layer, and where role-based function access is enforced. Method: manual code review plus this lab's own **isolation-tester** harness (`isolation-tester/`), which replays every declared endpoint as a foreign tenant, a lower role, a spoofed header, and a mass-assigned tenant id, then classifies each response as LEAK or DENIED.

Architecture at a glance

- **Tenancy model:** shared schema, `tenant_id` column on every tenant-scoped table.
- **AuthN:** Keycloak-issued JWTs (resource server pattern).
- **Tenant resolution (as found):** an `X-Tenant-Id` request header, not the JWT's own claim - see Finding A-04, the root-cause enabler behind most other findings below.
- **Data access:** direct repository calls by primary key, with tenant scoping left to each call site rather than enforced centrally (no Postgres RLS in the baseline).

Top findings (Starter tier - 9 of 13 total; see the Standard/Advanced tier report for the complete finding set)

A-01 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** harness (`isolation-tester`)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:45`

GET /invoices/{id} loads the invoice by primary key alone - it never checks that the invoice belongs to the caller's own tenant. Any authenticated user from ANY tenant can read any other tenant's invoice by guessing or enumerating sequential ids - full cross-tenant financial data disclosure (line items, totals, customer names).

Fix: Load by (id, tenant_id) together - either a repository method `findByIdAndTenantId`, or a Postgres RLS policy comparing tenant_id to a session-local `app.tenant_id` (see targets/tenant-api-fixed).

A-02 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/customer/CustomerController.java:48`

PUT /customers/{id} updates the customer row by id alone, with no tenant check on the write path. Any authenticated user from any tenant can OVERWRITE another tenant's customer record - not just read, but corrupt another company's data.

Fix: Same fix pattern as A-01, applied to the write path: `findByIdAndTenantId` before the update.

A-03 — High

OWASP: A01:2021 - Broken Access Control · API3:2023 - Broken Object Property Level Authorization · **How found:** harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:59`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceCreateRequest.java:8`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceResponse.java:6`

The invoice JSON exposes internal fields (tenantId, cost fields) the client should never see or set, and POST /invoices lets the client set tenantId directly on create (mass assignment). A malicious or compromised client can create an invoice tagged as belonging to a DIFFERENT tenant, planting data in another company's account; the exposed internal fields leak cost/margin data never meant for the API consumer.

Fix: Introduce request/response DTOs that never carry tenantId; derive tenantId server-side from the caller's own token, never from the request body.

A-04 — Critical

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/customer/CustomerController.java:27`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:33`
- `src/main/java/com/ledgerlite/tenantapi/tenant/TenantContext.java:13`

The tenant context is resolved from an **X-Tenant-Id** HTTP header instead of the verified JWT's own tenant claim. Any caller can set this header to an arbitrary value and impersonate any tenant for every request - this is the root-cause enabler behind most of the OTHER cross-tenant findings, since it means "which tenant is this request for" is entirely client-controlled.

Fix: Resolve tenant_id from the JWT's own verified claim only; ignore any client-supplied header for this purpose entirely.

A-05 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/export/ExportController.java:42`
- `src/main/java/com/ledgerlite/tenantapi/export/ExportService.java:16`

Report exports are written to a shared temp path keyed by a sequential integer id, with no ownership check on download. An attacker can walk sequential export ids and download OTHER tenants' exported invoice reports - a bulk, offline variant of the same cross-tenant data leak as A-01.

Fix: Check export ownership (`tenant_id`) before serving; use non-sequential (UUID) export ids as defence in depth.

A-06 — High

OWASP: A01:2021 - Broken Access Control · API1:2023 - Broken Object Level Authorization · **How found:** harness (isolation-tester)

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/dashboard/DashboardController.java:13`

The dashboard summary cache key does not include the tenant, so cached totals from one tenant are served to the next. A tenant can see another tenant's dashboard revenue/invoice totals - cross-tenant data leak via a shared cache, intermittent and timing-dependent (harder to notice than a direct API leak, easy to miss in a manual review).

Fix: Key the cache entry on `tenant_id` (e.g. `summary:{tenantId}`), never a constant string.

A-09 — High

OWASP: A05:2021 - Security Misconfiguration · API8:2023 - Security Misconfiguration · **How found:** tool (gitleaks)

Evidence:

- `src/main/resources/application.yml:8`

The JWT signing secret and the database password are hardcoded in the committed `application.yml`. Anyone with read access to the source repository (including a former employee, a leaked backup, or a public-repo mistake) obtains credentials that can forge auth tokens or connect to the database directly.

Fix: Read both from environment variables with no hardcoded fallback; the app should fail to start if they're unset, not silently use a default.

A-12 — High

OWASP: A03:2021 - Injection · API8:2023 - Security Misconfiguration · **How found:** manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceController.java:74`
- `src/main/java/com/ledgerlite/tenantapi/invoice/InvoiceSearchRepository.java:9`

The invoice search endpoint's `sort` query parameter is concatenated directly into the SQL ORDER BY clause. Classic SQL injection via the sort parameter - and NOT meaningfully contained by the query's separately-parameterised WHERE `tenant_id` clause: a payload like a CASE WHEN subquery in the ORDER BY

position turns this into a boolean-blind oracle that can read any table the database role can see, tenant-scoped or not, one bit at a time, and (depending on the JDBC driver's statement-batching config) may permit stacked statements. Treat this as full database read access, not a tenant-scoped leak.

Fix: Whitelist `sort` against a fixed set of known column names server-side before it ever reaches the query; never string-concatenate user input into SQL.

A-13 — High

OWASP: A10:2021 - Server-Side Request Forgery · API7:2023 - Server Side Request Forgery · **How found:** manual

Evidence:

- `src/main/java/com/ledgerlite/tenantapi/webhook/WebhookController.java:12`

The webhook 'test' endpoint fetches whatever URL the caller provides, including internal/metadata IP ranges. Classic SSRF - an attacker can use this endpoint to probe internal network services or reach the cloud metadata endpoint (169.254.169.254) and potentially exfiltrate instance credentials.

Fix: Require https, enforce a per-deployment host allow-list, reject IP literals and private/link-local ranges, disable redirects, and validate BEFORE the outbound call is made.

Risk summary

Severity	Count
Critical	3
High	6
Medium	4
Low	0
Total	13

Full findings, evidence, remediation plan and retest notes: `SECURITY_REVIEW_FULL.md` (Standard/Advanced tier).